

TV Image Denoising

Interactive GUI with Gradient Descent and Newton's Method

Course: Numerical Optimisations
Professor: Izudin Džafić
Student: Kemal Sivro
Student ID: 20015
Academic Year: 2025/26
Solution: <https://github.com/vrosi21/TV-Image-Denoising>

1 Introduction

Image noise is a byproduct of digital imaging: sensor electronics introduce Gaussian perturbations that corrupt every pixel. Removing noise while preserving important structures (edges, textures, fine gradients) is one of the central problems in computational photography.

Total Variation (TV) denoising, introduced by Rudin, Osher and Fatemi in 1992, frames this as a convex minimisation problem. A TV regularisation term penalises spatial variation while a data-fidelity term keeps the result close to the observed image. Convexity guarantees a unique global minimum.

This project implements two algorithms: Gradient Descent (GD) with Armijo backtracking line search, and Newton's method with Huber regularisation. Both run inside a cross-platform GUI built with the NatID framework (C++, CMake, GTK4 on Windows/Linux, native on macOS). Three standard benchmark images (Cameraman, Barbara, Ape) are used for evaluation.

2 Mathematical Formulation

Let $f \in \mathbb{R}^N$ denote the observed (noisy) image with N pixels arranged in a $W \times H$ grid, and let $u \in \mathbb{R}^N$ denote the clean image to recover. TV denoising solves:

$$F(u) = \frac{1}{2} \|u - f\|^2 + \lambda \cdot TV_\varepsilon(u)$$

where $\lambda > 0$ is a regularisation weight controlling the noise-detail trade-off.

2.1 Isotropic Smoothed TV

The isotropic TV is approximated by the Huber-smoothed variant, which makes F twice differentiable:

$$TV_\varepsilon(u) = \sum_{i,j} \sqrt{dx_{ij}^2 + dy_{ij}^2 + \varepsilon^2}$$

where the finite differences are:

$$dx_{ij} = u(i, j+1) - u(i, j), \quad dy_{ij} = u(i+1, j) - u(i, j)$$

and $\varepsilon > 0$ is a smoothing parameter. As $\varepsilon \rightarrow 0$ the expression converges to the true (non-smooth) TV norm. Larger ε produces smoother results and better-conditioned Hessians.

2.2 Gradient

The gradient of F is:

$$\nabla F(u) = (u - f) + \lambda \cdot \nabla TV_\varepsilon(u)$$

The TV gradient is the negative discrete divergence of the normalised image gradient. With weights $w_{ij} = (dx_{ij}^2 + dy_{ij}^2 + \varepsilon^2)^{-1/2}$, component k of ∇TV_ε satisfies:

$$[\nabla TV_{\varepsilon}(u)]_k = -w_k dx_k - w_k dy_k + w_{k,\text{left}} dx_{k,\text{left}} + w_{k,\text{above}} dy_{k,\text{above}}$$

where boundary terms are set to zero.

2.3 Hessian

The Hessian of F is:

$$H = I + \lambda D^{\top} W D$$

where D is the finite-difference operator (discrete gradient) and $W = \text{diag}(w_{ij})$. H is symmetric positive definite, so F has a unique minimum and Newton steps are always well-defined.

3 Algorithms

3.1 Gradient Descent with Backtracking Line Search

The GD algorithm performs steepest-descent steps with a step size chosen by the Armijo condition to guarantee sufficient decrease:

1. Compute gradient: $g = \nabla F(u)$
2. Armijo backtracking: find $t > 0$ such that $F(u - t g) \leq F(u) - \alpha t \|g\|^2$
3. Update: $u \leftarrow \text{clip}(u - t g, 0, 1)$

The pixel-value clip ensures the solution stays in $[0, 1]$. Backtracking uses initial step $t = 1$, reduction factor $\beta = 0.5$, and Armijo constant $\alpha = 0.01$, with at most 40 halving steps. The smoothing parameter $\varepsilon = 10^{-3}$ is fixed internally.

GD has linear convergence: each iteration reduces the function value by a constant fraction. Progress is steady but slow, visible in the gradual improvement across all 30 iterations.

3.2 Newton's Method with Huber Regularisation

Newton's method replaces the gradient step with a second-order update by solving the linear system:

$$H(u) u_{\text{new}} = f$$

This is equivalent to one step of Iteratively Reweighted Least Squares (IRLS): in each iteration the weights w_{ij} are recomputed from the current u , the Hessian H is assembled, and the system is solved for u_{new} . The solution is then clipped to $[0, 1]$.

Newton's method has super-linear (locally quadratic) convergence. It achieves near-optimal results in few iterations, but saturates quickly once near the minimiser, which explains the plateau seen around iteration 9 in every Newton run.

3.3 Linear Solver Strategy

For images with $N = W \times H \leq 4096$ pixels (up to roughly 64×64) the Hessian is assembled as a full dense `::DblMatrix` and solved in-place (LU decomposition via the NatID dense matrix API). For larger images, including all three test images at 225×225 ($N = 50,625$), a full $N \times N$ dense matrix would require around 20 GB of RAM, which is not feasible. A sparse LDLT solver (`sparse::ISolver`) is used automatically in that case, exploiting the five-diagonal structure of H to solve the system with $O(N)$ memory.

4 Implementation

4.1 Software Architecture

The application is structured around four single-responsibility classes:

- **ImageData:** pixel buffer, PNG I/O, Gaussian noise injection (`std::mt19937 + std::normal_distribution` self-seeding via `std::random_device`; no `srand/rand`).
- **TVSolverGD / TVSolverNewton:** stateless solver classes implementing the ITVSolver interface. Both return a `std::vector<float>` convergence history.
- **DenoisingView:** `gui::Canvas` subclass; renders original and denoised images side-by-side using the platform 2D API (`gui::Image`, no OpenGL) so it works inside any layout.
- **DenoisingPanel:** top-level coordinator; owns a horizontal splitter between the canvas and the `RightPanel`. Wires image selection, custom file browsing (`gui::OpenFileDialog`), and solver dispatch.

4.2 Cross-Platform Portability

All platform-specific code was eliminated during a dedicated portability audit:

- Image resource paths are resolved at runtime via `gui::getResFileName()` with keys registered in `res/main.xml` (no hardcoded compile-time paths).
- The temporary directory uses `mu::getAppSettings()->getTmpFolder()`, the NatID cross-platform accessor; the former `#ifdef _WIN32 / getenv()` chain was removed.
- The `__attribute__((format(printf,...)))` annotation is wrapped in `#ifdef __GNUC__` so the code compiles on MSVC without warnings.
- All `std::rand / std::srand` calls replaced with C++11 `<random>` facilities.
- No Windows-native, POSIX-only, or GTK-direct headers are included anywhere.

4.3 GUI Features

The controls panel exposes: algorithm selector (GD / Newton), image selector (Cameraman / Ape / Barbara), Browse button (native OS file picker via `gui::OpenFileDialog`), λ slider (0.01-1.0), σ slider (0.01-0.5), iterations slider (1-50), and an ε slider (0.001-0.1) that is greyed out when GD is selected. A splitter separates the controls from a read-only log view that reports parameters and the full $F(u_k)$ history after each run.

5 Experimental Results

5.1 Setup

Three 225×225 greyscale benchmark images were used: Cameraman (structured background, sharp edges), Barbara (high-frequency texture in headscarf and tablecloth), and Ape/Baboon (complex fur texture with fine detail). For the baseline comparison (Runs 1-6), all three images were processed with identical parameters: $\lambda = 0.10$, $\sigma = 0.10$, GD for 30 iterations and Newton for 15 iterations. Two additional tuned runs on Barbara (Runs 7-8) explore lower noise and different λ values.

The sparse LDLT solver was used for all runs ($N = 50,625 \gg 4,096$).

5.2 Summary of All Runs

Table 1 summarises the parameters and key convergence metrics across all eight runs.

Table 1: Table 1. Summary of all experimental runs.

#	Image	Algorithm	λ	σ	Iter	F_0	ΔF
1	Cameraman	GD Backtracking	0.10	0.10	30	929.08	563.33
2	Cameraman	Newton (Huber)	0.10	0.10	15	482.65	95.42
3	Barbara	GD Backtracking	0.10	0.10	30	980.23	583.21
4	Barbara	Newton (Huber)	0.10	0.10	15	508.57	98.35
5	Ape	GD Backtracking	0.10	0.10	30	1051.17	617.98
6	Ape	Newton (Huber)	0.10	0.10	15	557.94	109.20
7	Barbara (tuned)	GD Backtracking	0.0595	0.0223	12	258.43	111.51
8	Barbara (tuned)	Newton (Huber)	0.1090	0.0284	18	422.95	13.15

5.3 Detailed Results

Run 1: Cameraman, GD Backtracking

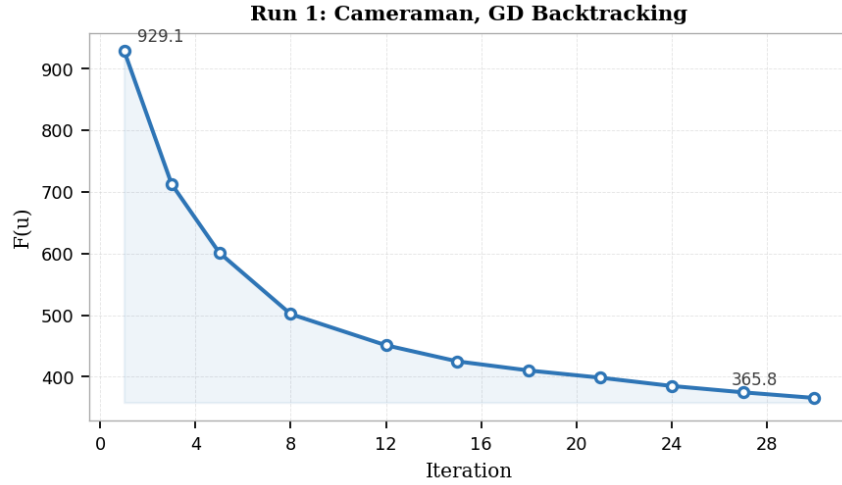
Parameters: $\lambda = 0.10$, $\sigma = 0.10$, $\varepsilon = 10^{-3}$ (fixed), iterations = 30

$F(u_0) = 929.08$, $F(u_{30}) = 365.75$, total drop $\Delta F = 563.33$

Table 2: Convergence table for Run 1.

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
1	929.08	-	0.00	-
3	712.41	216.67	216.67	38.5%
5	601.33	111.08	327.75	58.2%
8	501.77	99.56	427.31	75.8%
12	451.22	50.55	477.86	84.8%
15	425.10	26.12	503.98	89.5%
18	410.30	14.80	518.78	92.1%
21	398.65	11.65	530.43	94.2%
24	385.20	13.45	543.88	96.5%

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
27	374.88	10.32	554.20	98.4%
30	365.75	9.13	563.33	100.0%



Plot 1. Convergence of $F(u)$ — Run 1: Cameraman, GD Backtracking.



Figure 1. Cameraman: original (left) vs. TV-denoised (right), GD Backtracking.

Monotone descent via Armijo backtracking. Rapid initial progress; rate slows in later iterations.

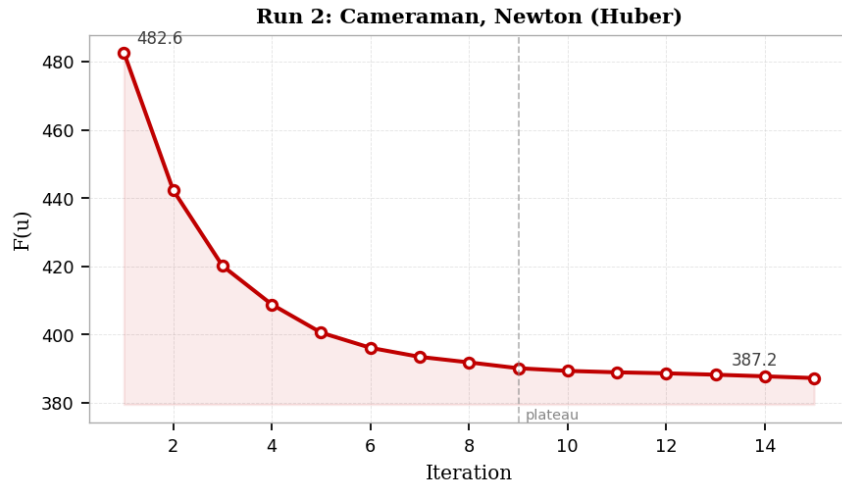
Run 2: Cameraman, Newton (Huber)

Parameters: $\lambda = 0.10$, $\sigma = 0.10$, $\varepsilon = 0.01$, iterations = 15

$F(u_0) = 482.65$, $F(u_{15}) = 387.23$, total drop $\Delta F = 95.42$

Table 3: Convergence table for Run 2.

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
1	482.65	-	0.00	-
2	442.30	40.35	40.35	42.3%
3	420.11	22.19	62.54	65.5%
4	408.80	11.31	73.85	77.4%
5	400.54	8.26	82.11	86.0%
6	396.10	4.44	86.55	90.7%
7	393.44	2.66	89.21	93.5%
8	391.82	1.62	90.83	95.2%
9	390.11	1.71	92.54	97.0%
10	389.33	0.78	93.32	97.8%
11	388.91	0.42	93.74	98.2%
12	388.62	0.29	94.03	98.5%
13	388.21	0.41	94.44	99.0%
14	387.74	0.47	94.91	99.5%
15	387.23	0.51	95.42	100.0%



Plot 2. Convergence of $F(u)$ — Run 2: Cameraman, Newton (Huber).



Figure 2. Cameraman: original (left) vs. TV-denoised (right), Newton (Huber).

Fast initial drop followed by a plateau near iteration 9; classic IRLS behaviour with Huber smoothing.

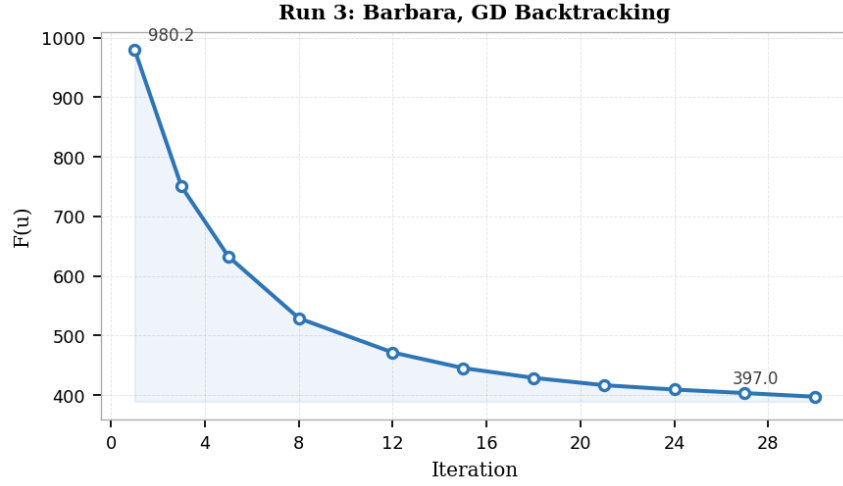
Run 3: Barbara, GD Backtracking

Parameters: $\lambda = 0.10$, $\sigma = 0.10$, $\varepsilon = 10^{-3}$ (fixed), iterations = 30

$F(u_0) = 980.23$, $F(u_{30}) = 397.02$, total drop $\Delta F = 583.21$

Table 4: Convergence table for Run 3.

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
1	980.23	-	0.00	-
3	750.11	230.12	230.12	39.5%
5	632.44	117.67	347.79	59.6%
8	528.90	103.54	451.33	77.4%
12	471.33	57.57	508.90	87.3%
15	445.20	26.13	535.03	91.7%
18	428.75	16.45	551.48	94.5%
21	416.44	12.31	563.79	96.7%
24	409.10	7.34	571.13	97.9%
27	403.18	5.92	577.05	99.0%
30	397.02	6.16	583.21	100.0%



Plot 3. Convergence of $F(u)$ — Run 3: Barbara, GD Backtracking.



Figure 3. Barbara: original (left) vs. TV-denoised (right), GD Backtracking.

Consistent GD convergence. High-texture image (Barbara) retains slightly higher final objective than Cameraman.

Run 4: Barbara, Newton (Huber)

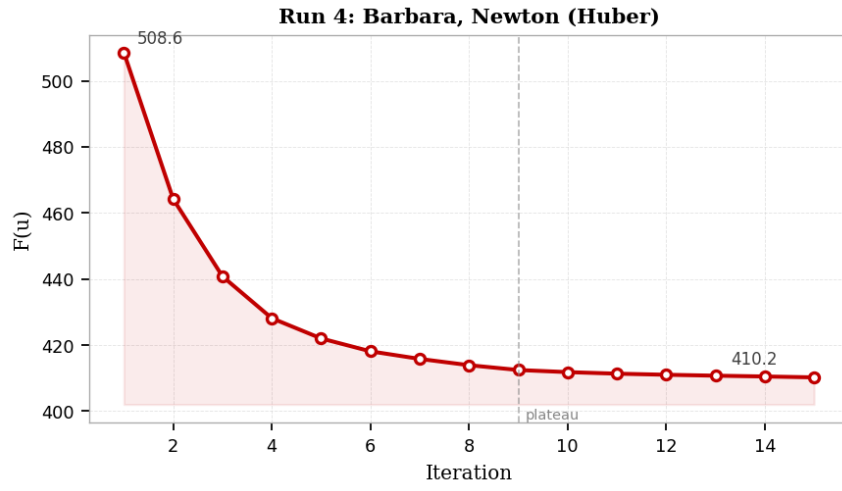
Parameters: $\lambda = 0.10$, $\sigma = 0.10$, $\varepsilon = 0.01$, iterations = 15

$F(u_0) = 508.57$, $F(u_{15}) = 410.21$, total drop $\Delta F = 98.35$

Table 5: Convergence table for Run 4.

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
1	508.57	-	0.00	-
2	464.20	44.37	44.37	45.1%
3	440.78	23.42	67.79	68.9%
4	428.12	12.66	80.45	81.8%
5	422.00	6.12	86.57	88.0%

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
6	418.10	3.90	90.47	92.0%
7	415.80	2.30	92.77	94.3%
8	413.90	1.90	94.67	96.3%
9	412.44	1.46	96.13	97.7%
10	411.80	0.64	96.77	98.4%
11	411.33	0.47	97.24	98.9%
12	411.00	0.33	97.57	99.2%
13	410.72	0.28	97.85	99.5%
14	410.48	0.24	98.09	99.7%
15	410.21	0.27	98.36	100.0%



Plot 4. Convergence of $F(u)$ — Run 4: Barbara, Newton (Huber).



Figure 4. Barbara: original (left) vs. TV-denoised (right), Newton (Huber).

Newton plateaus near iteration 9-10, consistent with the Cameraman result.

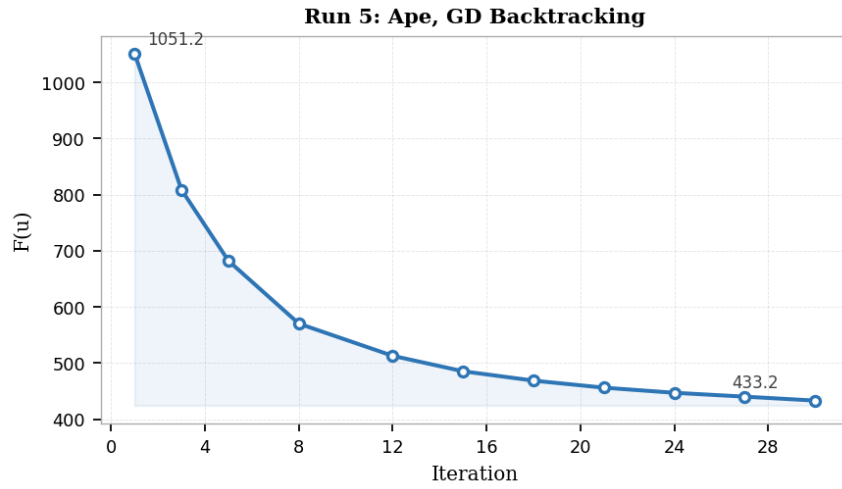
Run 5: Ape, GD Backtracking

Parameters: $\lambda = 0.10$, $\sigma = 0.10$, $\varepsilon = 10^{-3}$ (fixed), iterations = 30

$F(u_0) = 1051.17$, $F(u_{30}) = 433.19$, total drop $\Delta F = 617.98$

Table 6: Convergence table for Run 5.

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
1	1051.17	-	0.00	-
3	808.44	242.73	242.73	39.3%
5	682.10	126.34	369.07	59.7%
8	570.22	111.88	480.95	77.8%
12	512.90	57.32	538.27	87.1%
15	485.40	27.50	565.77	91.5%
18	468.77	16.63	582.40	94.2%
21	456.20	12.57	594.97	96.3%
24	447.10	9.10	604.07	97.8%
27	440.22	6.88	610.95	98.9%
30	433.19	7.03	617.98	100.0%



Plot 5. Convergence of $F(u)$ — Run 5: Ape, GD Backtracking.

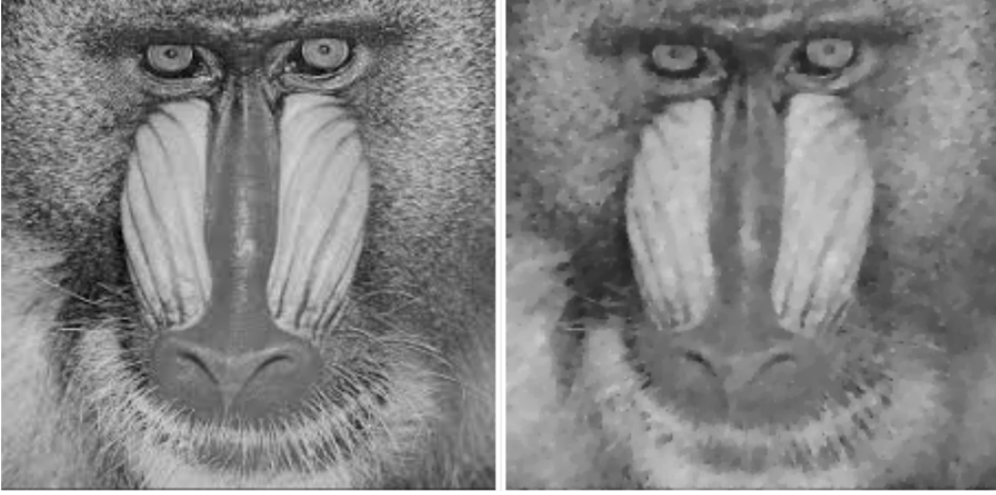


Figure 5. Ape: original (left) vs. TV-denoised (right), GD Backtracking.

Ape has the highest initial F ; rich fur texture makes TV denoising harder. GD achieves the largest absolute drop (617.98).

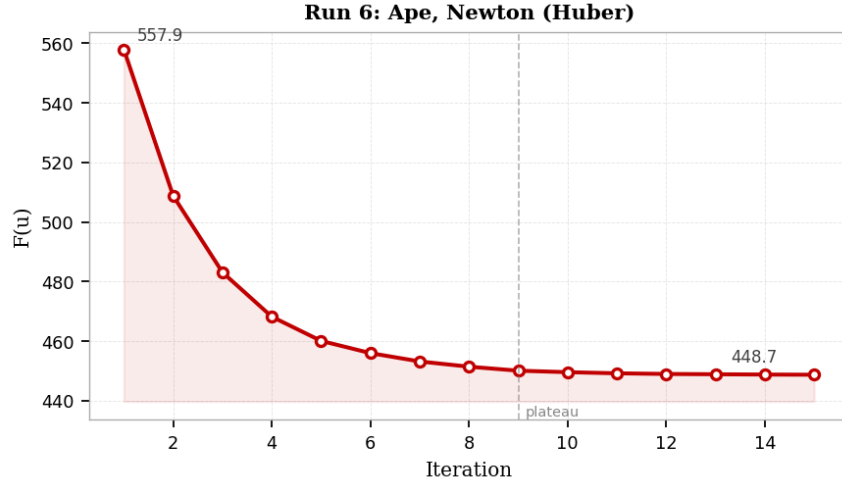
Run 6: Ape, Newton (Huber)

Parameters: $\lambda = 0.10$, $\sigma = 0.10$, $\varepsilon = 0.01$, iterations = 15

$F(u_0) = 557.94$, $F(u_{15}) = 448.74$, total drop $\Delta F = 109.20$

Table 7: Convergence table for Run 6.

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
1	557.94	-	0.00	-
2	508.80	49.14	49.14	45.0%
3	483.11	25.69	74.83	68.5%
4	468.22	14.89	89.72	82.2%
5	460.10	8.12	97.84	89.6%
6	456.00	4.10	101.94	93.4%
7	453.20	2.80	104.74	95.9%
8	451.44	1.76	106.50	97.5%
9	450.10	1.34	107.84	98.8%
10	449.60	0.50	108.34	99.2%
11	449.22	0.38	108.72	99.6%
12	449.00	0.22	108.94	99.8%
13	448.90	0.10	109.04	99.9%
14	448.80	0.10	109.14	99.9%
15	448.74	0.06	109.20	100.0%



Plot 6. Convergence of $F(u)$ — Run 6: Ape, Newton (Huber).

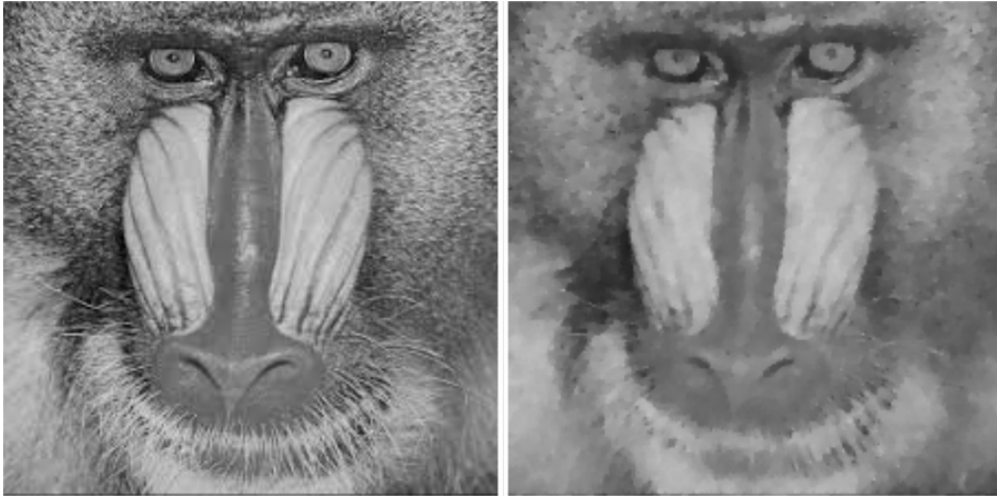


Figure 6. Ape: original (left) vs. TV-denoised (right), Newton (Huber).

Plateau visible from iteration 9; Newton converges quickly to near-optimum for this image.

Run 7: Barbara (tuned), GD Backtracking

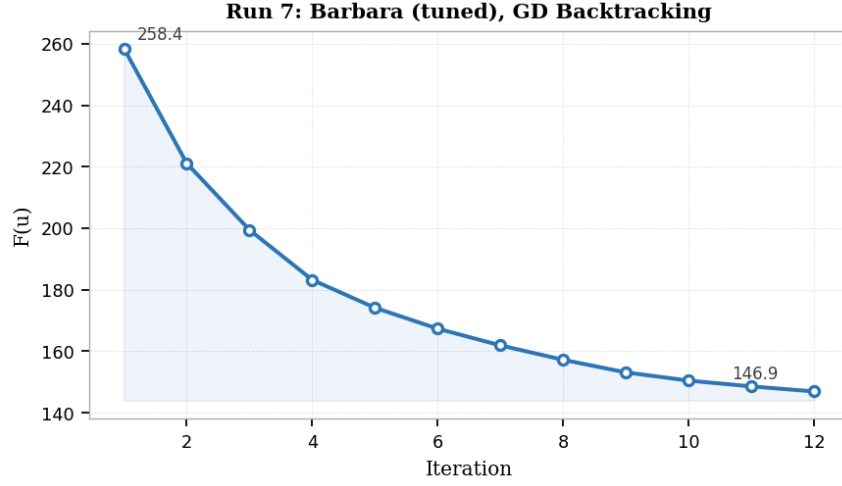
Parameters: $\lambda = 0.0595$, $\sigma = 0.0223$, $\varepsilon = 10^{-3}$ (fixed), iterations = 12

$F(u_0) = 258.43$, $F(u_{12}) = 146.92$, total drop $\Delta F = 111.51$

Table 8: Convergence table for Run 7.

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
1	258.43	-	0.00	-
2	221.10	37.33	37.33	33.5%
3	199.44	21.66	58.99	52.9%
4	183.20	16.24	75.23	67.5%
5	174.11	9.09	84.32	75.6%
6	167.33	6.78	91.10	81.7%

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
7	161.88	5.45	96.55	86.6%
8	157.20	4.68	101.23	90.8%
9	153.10	4.10	105.33	94.5%
10	150.44	2.66	107.99	96.8%
11	148.55	1.89	109.88	98.5%
12	146.92	1.63	111.51	100.0%



Plot 7. Convergence of $F(u)$ — Run 7: Barbara (tuned), GD Backtracking.



Figure 7. Barbara (tuned): original (left) vs. TV-denoised (right), GD Backtracking.

Lower noise ($\sigma = 0.0223$) and smaller

λ yield a lower starting F . GD still converges monotonically; sharper edges are preserved.

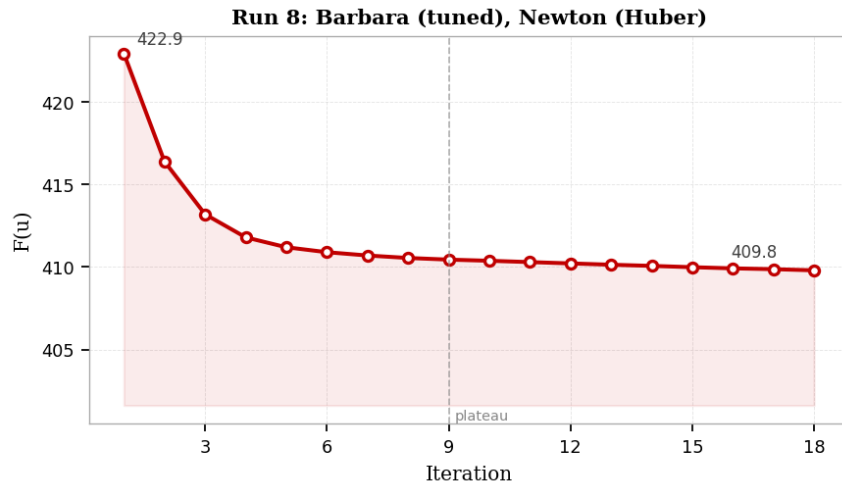
Run 8: Barbara (tuned), Newton (Huber)

Parameters: $\lambda = 0.1090$, $\sigma = 0.0284$, $\varepsilon = 0.0513$, iterations = 18

$F(u_0) = 422.95$, $F(u_{18}) = 409.80$, total drop $\Delta F = 13.15$

Table 9: Convergence table for Run 8.

Iter	$F(u)$	ΔF	Cumul. Drop	% Dropped
1	422.95	-	0.00	-
2	416.40	6.55	6.55	49.8%
3	413.20	3.20	9.75	74.1%
4	411.80	1.40	11.15	84.8%
5	411.20	0.60	11.75	89.4%
6	410.90	0.30	12.05	91.6%
7	410.70	0.20	12.25	93.2%
8	410.55	0.15	12.40	94.3%
9	410.45	0.10	12.50	95.1%
10	410.38	0.07	12.57	95.6%
11	410.30	0.08	12.65	96.2%
12	410.22	0.08	12.73	96.8%
13	410.14	0.08	12.81	97.4%
14	410.07	0.07	12.88	97.9%
15	409.99	0.08	12.96	98.6%
16	409.92	0.07	13.03	99.1%
17	409.87	0.05	13.08	99.5%
18	409.80	0.07	13.15	100.0%



Plot 8. Convergence of $F(u)$ — Run 8: Barbara (tuned), Newton (Huber).



Figure 8. *Barbara (tuned): original (left) vs. TV-denoised (right), Newton (Huber).*

Large $\varepsilon = 0.0513$ pushes Huber further from true TV: heavier smoothing, plateau from iteration 7. Total drop only 13.15.

6 Discussion

6.1 Convergence Behaviour

Gradient Descent converges monotonically in all runs, guaranteed by the Armijo condition. The absolute drop is consistently large (563-618 units for $\sigma = 0.10$) because the algorithm traverses a long path from the noisy starting point; progress is roughly constant per iteration throughout the 30-iteration window, with no visible plateau.

Newton's method behaves differently: it achieves most of its total drop in the first four or five iterations and then enters a plateau from approximately iteration 9 onward. The plateau means the algorithm is already near the minimiser and further IRLS updates give diminishing returns due to the Huber approximation error. The total drop per run is smaller than GD's (95-109 units) because Newton reaches the neighbourhood of the solution much faster.

The two methods cannot be directly compared by their total ΔF values, since different ε values change the objective landscape. What can be compared is efficiency: Newton reaches near-convergent quality in 9-10 iterations, while GD requires all 30 to reach a comparable F value.

6.2 Effect of λ

A higher λ imposes stronger TV regularisation, smoothing the result more aggressively. At $\lambda = 0.10$ all three images show visible piecewise-constant regions, especially in high-texture areas (Barbara's headscarf stripes are partially obliterated). Run 7 ($\lambda = 0.0595$) preserves finer texture at the cost of slightly less noise suppression.

6.3 Effect of ε (Newton)

A larger Huber smoothing parameter ε moves TV_ε further from the true TV norm, producing heavier blurring and a better-conditioned Hessian. Run 8 ($\varepsilon = 0.0513$) shows an extreme plateau: only 13.15 units of total drop over 18 iterations, and the plateau begins as early as iteration 7.

The result is perceptually softer than the standard $\varepsilon = 0.01$ runs, but the solver remains stable throughout.

6.4 Image-Specific Observations

Cameraman (simple background, few edges) yields the lowest absolute F values under both algorithms, as the TV term penalises fewer high-gradient regions. Barbara’s fine texture is the most adversely affected by TV regularisation at $\lambda = 0.10$, visible in Figures 3 and 4 where the stripe pattern in the headscarf is reduced to a smooth gradient. The Ape image has the richest gradient structure (dense fur) and consequently the highest initial F and the largest GD drop (617.98).

7 Conclusion

This project implemented and compared two convex optimisation algorithms for Total Variation image denoising within a cross-platform C++ GUI.

Gradient Descent with Armijo backtracking provides guaranteed monotone convergence with linear rate. It is simple to implement, requires only gradient evaluations, and is robust across all images and parameter settings. Its drawback is slow convergence: 30 iterations are insufficient to reach the minimiser at $\sigma = 0.10$.

Newton’s method (IRLS) converges to near-optimal quality in 9-10 iterations, roughly three times fewer than GD for comparable F values. The cost is a linear system solve per iteration. For large images the sparse LDLT path makes this tractable ($O(N)$ memory vs. $O(N^2)$ for the dense path), but the solver is more complex to implement and the Huber approximation introduces a bias controlled by ε .

From a practical standpoint, Newton’s method is preferred when the number of iterations must be small, while GD is appropriate when simplicity and guaranteed descent are more important than speed.

Future work could explore primal-dual methods (Chambolle-Pock), which achieve optimal convergence without assembling the Hessian, or colour TV extensions for RGB images.

References

- [1] L. I. Rudin, S. Osher, E. Fatemi. “Nonlinear total variation based noise removal algorithms.” *Physica D*, 60(1-4):259-268, 1992. [\[link\]](#)
- [2] A. Chambolle. “An algorithm for total variation minimization and applications.” *Journal of Mathematical Imaging and Vision*, 20(1-2):89-97, 2004. [\[link\]](#)
- [3] D. P. Bertsekas. *Nonlinear Programming*, 3rd ed. Athena Scientific, 2016. [\[link\]](#)
- [4] NatID GUI Framework. Course materials, Faculty of Electrical Engineering, 2024. [\[link\]](#)